

The Unified Media–Interface Architecture for BeamletOptics.jl

The Unified Media–Interface Architecture for BeamletOptics.jl

A First-Principles, Zero-Allocation Foundation for Geometric and Beamlet Optics

Document Version: 2.1 (Comprehensive Architectural Specification)
Target Branches: feature/intersection-handling, master
Author: Optical Architecture Working Group

1. Executive Summary & Paradigm Shift

Historically, BeamletOptics.jl (BMO) distributed optical physics across concrete optical components. Every component type (SphericalLens, Mirror, PlateBeamsplitter, PolarizationFilter, Detector) implemented its own custom interact3d method, duplicating Snell’s law, Fresnel equations, normal inversion, and ray creation. Furthermore, tracking multi-material interfaces (doublets, coatings, beamsplitters) relied on ad-hoc object mutation and tolerance searches (MultiIntersection).

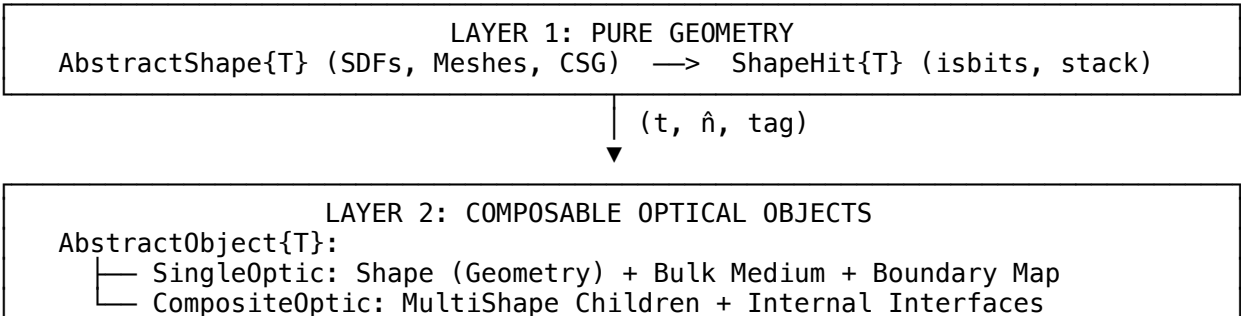
The Foundational Insight

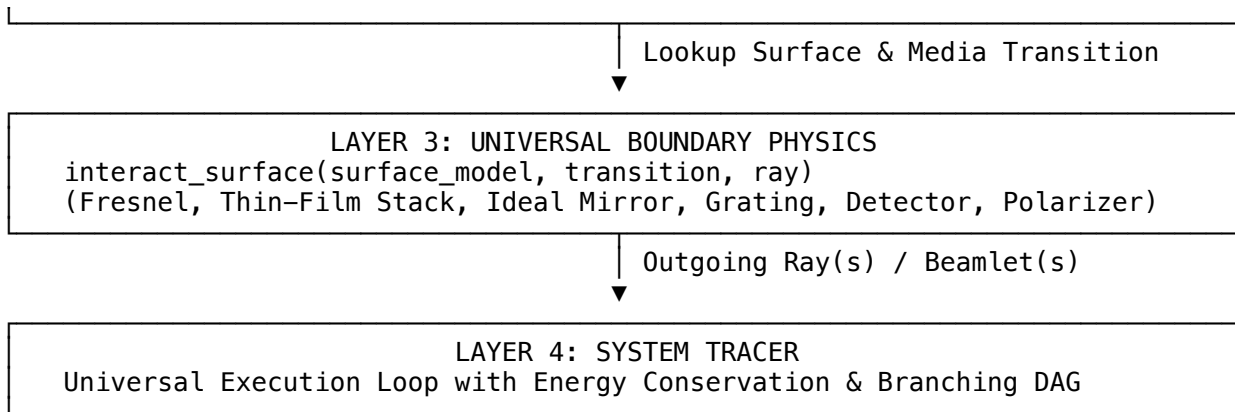
A lens is not an optical interaction; a lens is a geometric volume filled with a physical medium.

In natural optics: 1. **Geometry (Space & Boundaries):** SDFs and Meshes partition into spatial domains bounded by oriented manifolds. 2. **Media (Volumetric Physics):** Light propagates through bulk materials characterized by complex refractive indices or dielectric tensors. 3. **Interfaces (Boundary Physics):** When a wavepacket encounters an oriented boundary between Medium 1 and Medium 2, Maxwell’s boundary conditions uniquely dictate reflection, refraction, absorption, splitting, or diffraction.

By decomposing BMO into these three orthogonal layers: * **90% of component boilerplate is eliminated.** * The engine becomes **100% immutable and type-stable (zero GC pressure).** * Complex multi-element assemblies (doublets, prisms, coatings, gratings) work **universally and automatically.**

2. Architectural Overview: The Four Decoupled Layers





3. Layer 1: Pure Geometry & ShapeHit{T}

Geometric solvers (SDF ray-marchers, analytical shapes, BVH meshes) have zero awareness of refractive indices, wavelengths, or optical physics. They solve pure geometry and return a stack-allocated, isbits geometric hit token:

```

"""
    ShapeHit{T<:Real}

Pure geometric intersection result. 100% immutable, zero heap allocations (isbits).
"""
struct ShapeHit{T <: Real}
    t::T                # Distance parameter along ray (p = origin + t * dir)
    normal::Point3{T}   # Outward unit normal vector (|n̂| = 1)
    tag::Symbol          # Intrinsic surface identifier (:front, :back, :side, etc.)
end
  
```

3.1 Generic SDF Ray-Marcher (Written Once for All SDFs)

In BMO, all SDF volumes inherit from AbstractSDF. The ray-marching algorithm is implemented **once** in src/SDFs/AbstractSDF.jl and automatically powers all SDF primitives, lens shapes, and CSG unions:

```

# Defined ONCE in src/SDFs/AbstractSDF.jl for ALL SDF types (spheres, aspheres, cylinders,
# unions, etc.)
function intersect3d(shape::AbstractSDF{T}, ray::AbstractRay{T}) where {T}
    t_hit, hit_found = ray_march(shape, position(ray), direction(ray))
    !hit_found && return nothing

    p_hit = position(ray) + t_hit * direction(ray)
    normal = normal3d(shape, p_hit)

    # Intrinsic tag queried generically from the SDF
    tag = surface_tag(shape, p_hit, normal)

    return ShapeHit{T}(t_hit, normal, tag)
end
  
```

3.2 Canonical, Fully-Typed Intersection

When a shape hit is associated with an AbstractObject, it is wrapped in an immutable Intersection:

```

"""
    Intersection{T<:Real, S<:AbstractShape{T}, O<:AbstractObject{T}} <:
        AbstractIntersection{T}

Fully typed, immutable token representing the ray intersection.
"""
struct Intersection{T <: Real, S <: AbstractShape{T}, O <: AbstractObject{T}} <:
    AbstractIntersection{T}
    hit::ShapeHit{T}
    shape::S
    object::O
end

@inline Base.length(i::Intersection) = i.hit.t
@inline normal3d(i::Intersection)    = i.hit.normal
@inline surface_tag(i::Intersection) = i.hit.tag
@inline shape(i::Intersection)       = i.shape
@inline object(i::Intersection)      = i.object

```

4. Layer 2: Optical Media & Surface Models

4.1 Volumetric Media (AbstractMedium)

Bulk materials govern phase accumulation, group velocity, dispersion, and bulk absorption:

```

abstract type AbstractMedium end

struct Ambient <: AbstractMedium end # Wildcard resolving to System environment (e.g.
    Air/Vacuum)

struct IsotropicMedium{D, A} <: AbstractMedium
    name::Symbol
    dispersion::D # Function  $\lambda \rightarrow n(\lambda)$  (e.g. SellmeierFormula, Cauchy, Constant)
    attenuation::A # Function  $\lambda \rightarrow \alpha(\lambda)$  (bulk absorption coefficient [1/m])
end

refractive_index(::Ambient,  $\lambda$ ) = 1.0
refractive_index(m::IsotropicMedium,  $\lambda$ ) = m.dispersion( $\lambda$ )

```

4.2 Surface Models (AbstractSurfaceModel)

Surface models govern physical behavior *at* the boundary interface:

```

abstract type AbstractSurfaceModel end

struct FresnelInterface <: AbstractSurfaceModel end # Bare dielectric boundary (Snell +
    Fresnel)
struct IdealMirror <: AbstractSurfaceModel end # 100% specular reflection
struct AbsorbingSurface <: AbstractSurfaceModel end # Ray termination
    (apertures/baffles)
struct DetectorSurface{D} <: AbstractSurfaceModel # Records power, phase, E-field
    detector_data::D
end
struct CoatedSurface{C} <: AbstractSurfaceModel # AR, HR, Beamsplitter, Thin-film
    stack
    coating_model::C
end

```

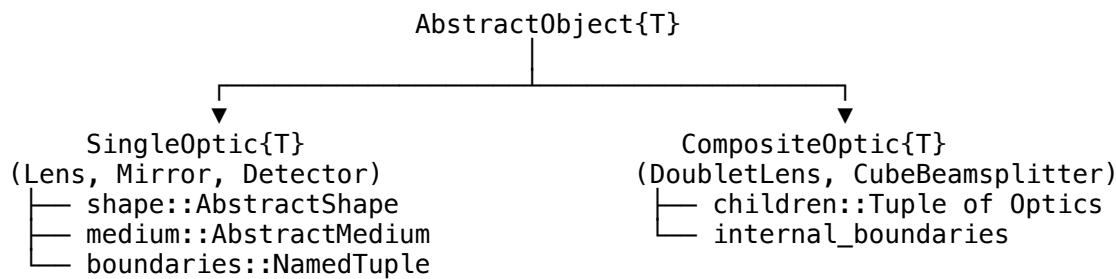
```

struct GratingSurface{T} <: AbstractSurfaceModel           # Diffraction grating:  $d * (\sin\theta_m - \sin\theta_i) = m\lambda$ 
    groove_density::T
    order::Int
end

```

5. Layer 3: The AbstractObject Type Hierarchy

All optical components in BMO are subtypes of `AbstractObject{T}`. We organize them into a clean, two-tier hierarchy:



5.1 Single Optics (SingleOptic <: AbstractObject)

A `SingleOptic` represents a single geometric volume (or 2D surface) possessing a bulk interior medium and boundary surface definitions:

```

abstract type SingleOptic{T} <: AbstractObject{T} end

```

```

# Default shape trait for single optics
shape_trait_of(::SingleOptic) = SingleShape()

```

```

# Standard generic implementation backing Lens, Mirror, Detector, etc.

```

```

struct Optic{T <: Real, S <: AbstractShape{T}, M <: AbstractMedium, B <: SingleOptic{T}
    shape::S
    medium::M
    boundaries::B # e.g. (front = CoatedSurface(AR), back = FresnelInterface(), side =
        AbsorbingSurface())
end

```

```

shape(o::Optic) = o.shape
medium(o::Optic) = o.medium
boundary(o::Optic, tag::Symbol) = getproperty(o.boundaries, tag)

```

User-Facing Constructors (Zero Boilerplate)

```

# Spherical Lens

```

```

function SphericalLens(r1, r2, thickness, diameter, glass; coatings=())
    s = SphericalLensSDF(r1, r2, thickness, diameter)
    m = IsotropicMedium(:Glass, glass, nothing)
    b = merge((front = FresnelInterface(), back = FresnelInterface(), side =
        AbsorbingSurface()), coatings)
    return Optic(s, m, b)
end

```

```

# Plano Mirror

```

```

function PlanoMirror(diameter, thickness; coating=IdealMirror())
    s = CylinderSDF(diameter/2, thickness)

```

```

    b = (front = coating, back = AbsorbingSurface(), side = AbsorbingSurface())
    return Optic(s, Ambient(), b)
end

# Detector
function Detector(width, height, resolution)
    s = RectangleSDF(width, height)
    b = (front = DetectorSurface(resolution), back = AbsorbingSurface())
    return Optic(s, Ambient(), b)
end

```

5.2 Composite Optics & the MultiShape Integration

A CompositeOptic (e.g. DoubletLens, CubeBeamsplitter) represents an assembly of multiple touching single optics that share internal boundaries.

```

abstract type CompositeOptic{T} <: AbstractObject{T} end

# Composite optics naturally implement the MultiShape trait
shape_trait_of(::CompositeOptic) = MultiShape()

```

The DoubletLens Struct & Constructor

```

struct DoubletLens{T, L1 <: SingleOptic{T}, L2 <: SingleOptic{T}, B} <: CompositeOptic{T}
    front_lens::L1
    back_lens::L2
    cemented_boundary::B # e.g. FresnelInterface() or CoatedSurface(CementLayer)
end

# MultiShape shape getter exposes constituent shapes for kinematics & intersection
shape(dl::DoubletLens) = (shape(dl.front_lens), shape(dl.back_lens))

```

How MultiShape Intersection & Kinematics Work

1. **Kinematics:** All spatial transforms (translate3d!, rotate3d!, align3d!) on a CompositeOptic automatically delegate across all child shapes via existing MultiShape trait methods.
2. **Intersection:** Testing a ray against a CompositeOptic queries all constituent shapes:

```

function intersect3d(::MultiShape, composite::CompositeOptic, ray::AbstractRay{R}) where {R}
    closest_hit::Nullable{ShapeHit{R}} = nothing
    closest_shape = nothing

    for child_shape in shape(composite)
        temp_hit = intersect3d(child_shape, ray)
        if temp_hit !== nothing
            if closest_hit === nothing || temp_hit.t < closest_hit.t
                closest_hit = temp_hit
                closest_shape = child_shape
            end
        end
    end

    isnothing(closest_hit) && return nothing
    # Returns Intersection preserving both the hit shape and the enclosing composite object
    return Intersection(closest_hit, closest_shape, composite)
end

```

3. **Boundary Transition in DoubletLens (Zero Hacks):** Because the Intersection stores both the hit shape (shape(dl.front_lens)) and the hit tag (:back), the doublet unambiguously resolves the internal transition:
- **Struck Front Face of Front Lens:** Ambient (Glass 1).
 - **Struck Cemented Interface:** (Glass 1) (Glass 2). Direct Snell refraction between the two glass indices without an intermediate air gap.
 - **Struck Rear Face of Back Lens:** (Glass 2) Ambient.
-

6. Layer 4: Universal Physics Dispatch Pipeline

Because optical physics is decoupled from component types, interact_surface is implemented **once per surface model**, not once per optical component.

6.1 Automatic Orientation & Media Transition Resolution

```
@inline function resolve_transition(optic::SingleOptic, ray::AbstractRay, normal::Point3,
    system_ambient)
    med_bulk = medium(optic) === Ambient() ? system_ambient : medium(optic)
    med_out = system_ambient

    if dot(direction(ray), normal) < 0
        # Incident from outside -> entering optic
        return (n_inc = refractive_index(med_out, wavelength(ray)),
            n_trans = refractive_index(med_bulk, wavelength(ray)),
            normal_eff = normal,
            entering = true)
    else
        # Incident from inside -> exiting optic
        return (n_inc = refractive_index(med_bulk, wavelength(ray)),
            n_trans = refractive_index(med_out, wavelength(ray)),
            normal_eff = -normal,
            entering = false)
    end
end
```

6.2 Universal Surface Interaction Methods

```
# Dielectric Fresnel Interface
function interact_surface(::FresnelInterface, trans, ray::PolarizedRay{T}) where {T}
    θi = angle3d(direction(ray), -trans.normal_eff)
    rs, rp, ts, tp = fresnel_coefficients(θi, trans.n_trans / trans.n_inc)

    if is_total_internal_reflection(rs, rp)
        new_dir = reflection3d(direction(ray), trans.normal_eff)
        new_pol = reflect_polarization(polarization(ray), rs, rp)
        n_out = trans.n_inc
    else
        new_dir, _ = refraction3d(direction(ray), trans.normal_eff, trans.n_inc,
            trans.n_trans)
        new_pol = refract_polarization(polarization(ray), ts, tp, trans.n_inc,
            trans.n_trans)
        n_out = trans.n_trans
    end

    new_pos = position(ray) + length(ray) * direction(ray)
    return PolarizedRay{T}(new_pos, new_dir, nothing, wavelength(ray), n_out, new_pol)
```

end

Specular Mirror Interface

```
function interact_surface(::IdealMirror, trans, ray::AbstractRay{T}) where {T}
    new_dir = reflection3d(direction(ray), trans.normal_eff)
    new_pos = position(ray) + length(ray) * direction(ray)
    return Ray{T}(new_pos, new_dir, nothing, wavelength(ray), trans.n_inc)
end
```

Absorptive / Detector Interface

```
function interact_surface(d::DetectorSurface, trans, ray::AbstractRay{T}) where {T}
    hit_pos = position(ray) + length(ray) * direction(ray)
    record_hit!(d.detector_data, hit_pos, ray)
    return nothing # Terminates ray cleanly
end
```

7. The Universal System Tracing Loop

With this architecture, the main tracing loop in `src/System.jl` becomes streamlined and universal:

```
function trace_step!(system::AbstractSystem, ray::AbstractRay{T}) where {T}
    # 1. Geometry phase: find closest intersection across all objects
    hit_intersection = trace_all(system, ray)
    isnothing(hit_intersection) && return nothing

    obj = object(hit_intersection)
    geom = hit_intersection.hit

    # 2. Transition phase: resolve incident/transmitted media and effective normal
    trans = resolve_transition(obj, ray, geom.normal, system.ambient)

    # 3. Physics phase: query boundary model and dispatch interaction
    surface_model = boundary(obj, geom.tag)
    return interact_surface(surface_model, trans, ray)
end
```

8. Comprehensive Comparison: Legacy vs. Definitive Architecture

Architectural Feature	Legacy BMO (master)	Initial PR #60 Draft	Definitive Architecture
Intersection Representation	Mutable struct with coincident pointers	Mutable hierarchy (MultiIntersection)	100% Immutable ShapeHit{T} (isbits)
Memory Allocation	Heap allocated per hit	Dynamic boxed references	Zero heap allocations (stack only)
Physics Dispatch	Duplicated in every component type	Duplicated in components + traits	Universal dispatch on (SurfaceModel, Medium)
Boundary Resolution	Heuristic isentering normal checks	Tolerant coincident - matching	Exact vector projection
Doublets & Cemented Interfaces	Coincident object pointer hacks	Coincident -search with tolerance	Native multi-material boundary
MultiShape Composites	Clobbered sub-shape references	Overwrite ObjectIntersection	Preserves constituent hit shape and face tag

Architectural Feature	Legacy BMO (master)	Initial PR #60 Draft	Definitive Architecture
Extensibility	New optic requires custom interact3d	New optic requires custom interact3d	New optic = Shape + Material + Boundary map

9. Direct Implementation Roadmap for feature/intersection-handling

- Step 1 (ShapeHit):** Define struct ShapeHit{T} in src/AbstractTypes/AbstractIntersection.jl. Update src/SDFs/AbstractSDF.jl and mesh solvers to return ShapeHit{T}.
- Step 2 (Intersection):** Replace ObjectIntersection, PlaneIntersection, and MultiIntersection with a single canonical Intersection{T, S, O}.
- Step 3 (Interface & Media):** Introduce AbstractMedium, AbstractSurfaceModel, and resolve_transition.
- Step 4 (Consolidate interact):** Replace monolithic interact3d implementations across Lenses.jl, Mirrors.jl, and Detectors.jl with the universal interact_surface pipeline.